

Project Documentation: Full Adder Module

Digital Design Lab

Project Overview

This lab implements a 1-bit **Full Adder** combinational logic circuit in Verilog HDL using continuous assignment statements (**assign**)[cite: 32]. The full adder computes the arithmetic sum of three 1-bit binary inputs: **a**, **b**, and an incoming carry bit **cin**[cite: 32]. It generates a 1-bit sum output (**sum**) and a carry-out output (**cout**)[cite: 32].

Specifications & Boolean Logic

- **Sum Equation:** Evaluated as the 3-input XOR combination[cite: 32]:

$$sum = a \oplus b \oplus cin$$

- **Carry-Out Equation:** Evaluated using standard majority logic[cite: 32]:

$$cout = (a \cdot b) + (a \cdot cin) + (b \cdot cin)$$

Port Summary

Port Name	Direction	Type	Width	Description
a	Input	wire	1	Operand A input bit[cite: 32]
b	Input	wire	1	Operand B input bit[cite: 32]
cin	Input	wire	1	Carry-in input bit[cite: 32]
sum	Output	wire	1	Arithmetic sum bit[cite: 32]
cout	Output	wire	1	Carry-out bit[cite: 32]

Table 1: Full Adder Port Interfaces

Verilog Source Code

Design Module (full_adder.v)

```
module full_adder(  
    input wire a,  
    input wire b,  
    input wire cin,  
    output wire sum,  
    output wire cout  
);  
  
    // Combinational logic assignments  
    assign sum = a ^ b ^ cin;
```

```
    assign cout = (a & b) | (a & cin) | (b & cin);

endmodule
```

Testbench Module (tb_full_adder.v)

```
'timescale 1ns / 1ps

module tb_full_adder;

    reg a;
    reg b;
    reg cin;

    wire sum;
    wire cout;

    // Instantiate Design Under Test (DUT)
    full_adder DUT (
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

    integer i;

    initial begin
        // Apply all 8 input combinations
        for (i = 0; i < 8; i = i + 1) begin
            {a, b, cin} = i[2:0];
            #10;
        end

        $finish;
    end

    // Monitor Output Display
    initial begin
        $monitor("Time = %0t | a = %b | b = %b | cin = %b | sum = %b | cout = %b",
            $time, a, b, cin, sum, cout);
    end

endmodule
```

Simulation & Verification

Execution Command

To run the simulation using Cadence Xcelium or ModelSim:

```
xrun full_adder.v tb_full_adder.v
```

Expected Simulation Output

The truth table simulation log confirms correct addition for all 8 input state vectors[cite: 32]:

Time = 0		a = 0		b = 0		cin = 0		sum = 0		cout = 0
Time = 10		a = 0		b = 0		cin = 1		sum = 1		cout = 0
Time = 20		a = 0		b = 1		cin = 0		sum = 1		cout = 0
Time = 30		a = 0		b = 1		cin = 1		sum = 0		cout = 1
Time = 40		a = 1		b = 0		cin = 0		sum = 1		cout = 0
Time = 50		a = 1		b = 0		cin = 1		sum = 0		cout = 1
Time = 60		a = 1		b = 1		cin = 0		sum = 0		cout = 1
Time = 70		a = 1		b = 1		cin = 1		sum = 1		cout = 1